TP1 : Comptage de mots

1- Version 1 : comptage séquentiel en Python

1-1 Principe

La première version est simple car le programme ouvre le fichier mots.txt et lit les mots avec split(). Ensuite, il utilise un dictionnaire pour compter combien de fois chaque mot apparaît. Si le mot existe déjà, on ajoute 1 à son compteur. Sinon, on commence à 1.

```
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ cat > wordcount.py <<'PY'
> def compter_occurrences(fichier):
    with open(fichier, "r", encoding="utf-8") as f:
        mots = f.read().split()

    compteur = {}
    for mot in mots:
        compteur[mot] = compteur.get(mot, 0) + 1
    return compteur

resultats = compter_occurrences("mots.txt")

for mot, nb in sorted(resultats.items(), key=lambda x: x[1], reverse=True):
    print(f"{mot} : {nb}")
```

Cette solution traite les mots dans un seul processus et charge le contenu du fichier en mémoire.

1-2 Exécution et résultat

Le programme a notamment affiché « chat : 24 », « chien : 11 », « arbre : 7 » et « fleur : 7 ». Le temps réel relevé dans le terminal est de 0,039 seconde

```
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ time python3 wordcount.py
chat : 24
chien : 11
arbre : 7
fleur : 7
montagne : 5
soleil : 5
maison : 5
livre : 4
ciel : 4
route : 3
mer : 3
lune : 3
oiseau : 2
jardin : 1
voiture : 1

real    0m0.039s
user    0m0.040s
sys     0m0.000s
```

2-Version 2 : pseudo-MapReduce en Python

2-1 Décomposition du traitement

La fonction mapper produit une paire (mot, 1) à chaque apparition. La fonction shuffle rassemble toutes les valeurs associées au même mot, puis reducer additionne ces valeurs et produit un nombre final pour chaque clé. Avec l' entrée « chat chien chat »

```
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ cat > mapreduce_wordcount.py <<'PY'
> from collections import defaultdict
  from collections import defaultdict
def mapper(ligne):
def mapper(ligne):rip().split()
    mots = ligne.strip().split()mots]
    return [(mot, 1) for mot in mots]
def shuffle(mapped_data):
def shuffle(mapped_data):(list)
    grouped = defaultdict(list)ata:
    for mot, valeur in mapped_data:
        grouped[mot].append(valeur)
    return grouped
def reducer(grouped_data):
def reducer(grouped_data):
    reduced = {}eurs in grouped_data.items():
    for mot, valeurs in grouped_data.items():
        reduced[mot] = sum(valeurs)
    return reduced
mapped = []
mapped = []
with open("mots.txt", "r", encoding="utf-8") as f:
with open("mots.txt", "r", encoding="utf-8") as f:
    for ligne in f:nd(mapper(ligne))
        mapped.extend(mapper(ligne))
grouped = shuffle(mapped)
grouped = shuffle(mapped)
reduced = reducer(grouped)
for mot, nb in sorted(reduced.items(), key=lambda x: x[1], reverse=True):
for mot, nb in sorted(reduced.items(), key=lambda x: x[1], reverse=True):
    print(f"{mot} : {nb}")
```

2-2 Exécution et résultat

Les résultats sont les mêmes que ceux de la première version avec 15 mots différents. Le temps d'exécution est de 0,021 seconde. Cette version est un peu plus rapide que la première, mais cela peut être dû au fonctionnement de l'ordinateur. Il faudrait faire plusieurs tests pour savoir laquelle est vraiment la plus rapide.

```
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ time python3 mapreduce_wordcount.py
chat : 24
chien : 11
arbre : 7
fleur : 7
montagne : 5
soleil : 5
maison : 5
livre : 4
ciel : 4
route : 3
mer : 3
lune : 3
oiseau : 2
jardin : 1
voiture : 1

real    0m0.021s
user    0m0.021s
sys     0m0.001s
```

3-Version 3 : Hadoop MapReduce

3-1 Rôle de HDFS et de YARN

La troisième version utilise Hadoop. Le fichier est d'abord envoyé dans HDFS, puis YARN et MapReduce s'occupent du traitement. Tout fonctionne sur une seule machine.

3-2 Dépôt du fichier et lancement

```
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ hdfs dfs -mkdir -p /user/$USER/wordcount/input
hdfs dfs -put mots.txt /user/$USER/wordcount/input/
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ hdfs dfs -ls /user/$USER/wordcount/input
Found 1 items
-rw-r--r--   1 taharamdan supergroup        503 2026-10-13 11:05 /user/taharamdan/wordcount/input/mots.txt
```

La vérification montre que le fichier mots.txt de 503 octets est dans notre HDFS. Ensuite, le programme WordCount de Hadoop a été lancé pour traiter ce fichier.

```
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ time hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.4.3.jar \
wordcount \
/user/$USER/wordcount/input \
/user/$USER/wordcount/output
2026-10-13 11:05:58,330 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-10-13 11:05:58,892 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/taharamdan/.staging/job_1791882203994_0001
2026-10-13 11:05:59,289 INFO input.FileInputFormat: Total input files to process : 1
2026-10-13 11:05:59,388 INFO mapreduce.JobSubmitter: number of splits:1
2026-10-13 11:05:59,792 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1791882203994_0001
2026-10-13 11:05:59,792 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-10-13 11:06:00,177 INFO conf.Configuration: resource-types.xml not found
2026-10-13 11:06:00,178 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-10-13 11:06:00,986 INFO impl.YarnClientImpl: Submitted application application_1791882203994_0001
2026-10-13 11:06:01,146 INFO mapreduce.Job: The url to track the job: http://LAPTOP-C314Q5B1.:8088/proxy/application_1791882203994_0001/
2026-10-13 11:06:01,147 INFO mapreduce.Job: Running job: job_1791882203994_0001
2026-10-13 11:06:13,631 INFO mapreduce.Job: Job job_1791882203994_0001 running in uber mode : false
2026-10-13 11:06:13,633 INFO mapreduce.Job:  map 0% reduce 0%
2026-10-13 11:06:22,784 INFO mapreduce.Job:  map 100% reduce 0%
2026-10-13 11:06:29,897 INFO mapreduce.Job:  map 100% reduce 100%
2026-10-13 11:06:30,915 INFO mapreduce.Job: Job job_1791882203994_0001 completed successfully
2026-10-13 11:06:31,049 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=198
                FILE: Number of bytes written=620717
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=630
                HDFS: Number of bytes written=126
                HDFS: Number of read operations=8
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
        Job Counters
                Launched map tasks=1
                Launched reduce tasks=1
                Data-local map tasks=1
                Total time spent by all maps in occupied slots (ms)=5780
                Total time spent by all reduces in occupied slots (ms)=4909
                Total time spent by all map tasks (ms)=5780
                Total time spent by all reduce tasks (ms)=4909
                Total vcore-milliseconds taken by all map tasks=5780
                Total vcore-milliseconds taken by all reduce tasks=4909
                Total megabyte-milliseconds taken by all map tasks=5918720
                Total megabyte-milliseconds taken by all reduce tasks=5026816
        Map-Reduce Framework
                Map input records=9
                Map output records=85
                Map output bytes=843
                Map output materialized bytes=190
                Input split bytes=127
                Combine input records=85
                Combine output records=15
                Reduce input groups=15
                Reduce shuffle bytes=190
                Reduce input records=15
                Reduce output records=15
                Spilled Records=30
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=102
                CPU time spent (ms)=2460
                Physical memory (bytes) snapshot=547598336
                Virtual memory (bytes) snapshot=5467588736
                Total committed heap usage (bytes)=365953024
                Peak Map Physical memory (bytes)=318881792
                Peak Map Virtual memory (bytes)=2728325120
                Peak Reduce Physical memory (bytes)=228716544
                Peak Reduce Virtual memory (bytes)=2739183616
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=503
        File Output Format Counters
                Bytes Written=126

real    0m35.473s
user    0m11.564s
sys     0m0.848s
```

Le programme s'est terminé avec succès. Les étapes Map et Reduce ont atteint 100 %. Hadoop affiche aussi les détails du traitement.

Compteur Hadoop	**Valeur**
Map input records	9 (lignes du fichier)
Map output records	85 (une sortie par mot)
Reduce input groups	15 (clés distinctes)
Reduce output records	15 (comptages finaux)
Launched map tasks	1
Launched reduce tasks	1
HDFS: Number of bytes written	126 octets

La sortie HDFS contient notamment « arbre 7 », « chat 24 », « chien 11 », « fleur 7 » et « voiture 1 ». Les lignes sont ordonnées par clé, ce qui explique la différence d'ordre d'affichage avec les deux scripts Python, triés par fréquence.



4- Résultats et comparaison

4-1 Concordance des comptages

Chaque approche devait retourner le même résultat sur le même fichier d'entrée. C'est bien ce qui a été observé. Le tableau suivant reprend les 15 valeurs vérifiées dans les sorties du terminal :

Mot	Occurrences	Mot	Occurrences	Mot	Occurrences
chat	24	chien	11	arbre	7
fleur	7	maison	5	soleil	5
montagne	5	livre	4	ciel	4
mer	3	lune	3	route	3
oiseau	2	jardin	1	voiture	1

La somme des occurrences vaut 85. Cela confirme que le comptage ne perd pas de mots et que les traitements sont cohérents entre eux.

4-2 Temps d'exécution mesurés

Approche	**Commande utilisée**	**Temps réel**
Python simple	time python3 wordcount.py	0,039 s
Pseudo-MapReduce	time python3 mapreduce_wordcount.py	0,021 s

Hadoop MapReduce	time hadoop jar ... wordcount ...	35,473 s

Pour ce petit fichier de 503 octets, Hadoop est beaucoup plus lent. Cela s'explique par le temps nécessaire pour lancer et organiser les différentes étapes du traitement.

Cela ne veut pas dire que Hadoop est toujours plus lent que Python. Ici, on a utilisé un petit fichier sur une seule machine. Hadoop prend plus de temps car il doit lancer plusieurs étapes. Mais pour de gros fichiers, il peut être plus utile. Pour mieux comparer les temps, il faudrait refaire les tests plusieurs fois.

5- Comparaison avec plusieurs tailles de fichiers

5-1 Génération des fichiers

Pour mieux comparer les trois méthodes, on a créé quatre fichiers de tailles différentes. Chaque ligne contient un seul mot. Nous avons commencé avec 100 lignes, puis répété ces lignes plusieurs fois pour obtenir des fichiers plus grands

```
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ python3 generation_fichiers.py
mots_100.txt : 100 lignes, 598 octets (m=1)
mots_1000.txt : 1000 lignes, 5980 octets (m=10)
mots_10000.txt : 10000 lignes, 59800 octets (m=100)
mots_100000.txt : 100000 lignes, 598000 octets (m=1000)
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ jps
13137 Jps
4534 NameNode
4662 SecondaryNameNode
4952 ResourceManager
4586 DataNode
5004 NodeManager
```

Fichier	Nombre de lignes	Taille
mots_100.txt	100	598 octets
mots_1000.txt	1 000	5 980 octets
mots_10000.txt	10 000	59 800 octets
mots_100000.txt	100 000	598 000 octets

5-2 Résultats des tests

Exécuter les trois méthodes sur chaque fichier et mesuré leur temps d'exécution :

```
taharamdan@LAPTOP-C314Q5B1:~/TP_WordCount$ python3 comparaison_tp.py

100 lignes (m=1)
wordcount.py : 0.0152 s
mapreduce_wordcount.py : 0.0150 s
Hadoop : lancement du job
Hadoop : 23.2939 s
Verification OK : 15 mots differents, 100 mots comptes.

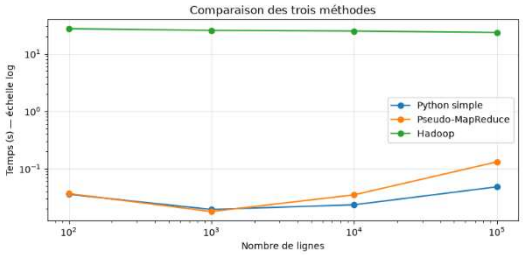
1000 lignes (m=10)
wordcount.py : 0.0177 s
mapreduce_wordcount.py : 0.0211 s
Hadoop : lancement du job
Hadoop : 22.3677 s
Verification OK : 15 mots differents, 1000 mots comptes.

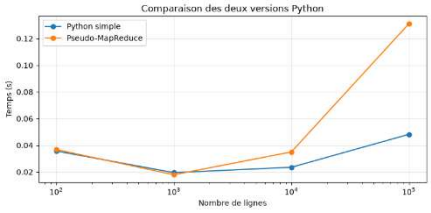
10000 lignes (m=100)
wordcount.py : 0.0217 s
mapreduce_wordcount.py : 0.0227 s
Hadoop : lancement du job
Hadoop : 23.2699 s
Verification OK : 15 mots differents, 10000 mots comptes.

100000 lignes (m=1000)
wordcount.py : 0.0399 s
mapreduce_wordcount.py : 0.0994 s
Hadoop : lancement du job
Hadoop : 22.7530 s
Verification OK : 15 mots differents, 100000 mots comptes.

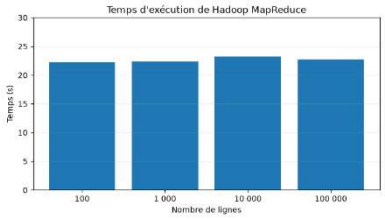
OK
```

Nombre de lignes	Python simple	Pseudo-MapReduce	Hadoop
100	0,0152 s	0,0150 s	22,2939 s
1 000	0,0177 s	0,0211 s	22,3677 s
10 000	0,0217 s	0,0227 s	23,2699 s
100 000	0,0399 s	0,0994 s	22,7530 s





Les trois méthodes donnent les mêmes résultats pour chaque fichier. Nous avons bien compté tous les mots, avec 15 mots différents



Le temps d'exécution de Hadoop reste presque le même pour les quatre fichiers. Cela montre que, dans nos tests, la taille du fichier change peu le temps de traitement.

5-3 Analyse

Les deux versions Python sont beaucoup plus rapides que Hadoop. Quand le fichier devient plus grand, Python prend un peu plus de temps, surtout avec pseudo-MapReduce. Pour Hadoop, le temps reste autour de 22 à 23 secondes, car il doit lancer plusieurs étapes avant de traiter le fichier.

6- Conclusion

Ce TP nous a permis de tester trois façons de compter les mots d'un fichier. Les trois donnent les mêmes résultats, mais Python est plus rapide pour les fichiers utilisés. Hadoop peut être plus utile pour traiter de gros fichiers sur plusieurs machines.